

OSSP (Operating Systems and Systems Programming) –25CS2104E

OSSP LAB – SKILL-3

Aim

To implement a simple command-line tokenizer and parser that splits user input into tokens, handles whitespace and delimiters, validates the input, detects syntax errors and empty commands, and produces a basic execution structure.

Algorithm

1. Start the program.
2. Read the command from the user.
3. Check whether the input is empty.
4. Identify delimiters such as spaces and tabs.
5. Split the input into individual tokens.
6. Store and display the tokens.
7. Identify the command and its arguments.
8. Validate the command syntax.
9. Detect invalid or empty input.
10. Display the parsed/execution structure.
11. Stop the program.

Task 1

```
bhavana@Bhavana:~$ ^C
bhavana@Bhavana:~$ nano tokenizer.c
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

struct Token {
    char type[20];
    char value[50];
};

int main() {
    char input[MAX];
    struct Token tokens[MAX];
    int count = 0;

    printf("Enter a command: ");
    fgets(input, MAX, stdin);

    char *word = strtok(input, " \\t\\n");

    while (word != NULL) {

        strcpy(tokens[count].value, word);

        if (strcmp(word, "ADD") == 0 ||
            strcmp(word, "SUB") == 0 ||
            strcmp(word, "MUL") == 0 ||
            strcmp(word, "DIV") == 0) {

            strcpy(tokens[count].type, "COMMAND");
        }

        else {
            int validNumber = 1;

            for (int i = 0; word[i] != '\\0'; i++) {

```

```

        for (int i = 0; word[i] != '\\0'; i++) {
            if (!isdigit(word[i])) {
                validNumber = 0;
                break;
            }
        }

        if (validNumber)
            strcpy(tokens[count].type, "NUMBER");
        else
            strcpy(tokens[count].type, "UNKNOWN");
    }

    count++;
    word = strtok(NULL, " \\t\\n");

    printf("\\nTokens:\\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d -> Type: %s, Value: %s\\n",
            i + 1,
            tokens[i].type,
            tokens[i].value);
    }

    if (count == 3 &&
        strcmp(tokens[0].type, "COMMAND") == 0 &&
        strcmp(tokens[1].type, "NUMBER") == 0 &&
        strcmp(tokens[2].type, "NUMBER") == 0) {

        printf("\\nToken stream is VALID.\\n");
    }
    else {
        printf("\\nToken stream is INVALID.\\n");
    }
}

```

```
bhavana@Bhavana:~$ gcc tokenizer.c -o tokenizer
bhavana@Bhavana:~$ ./tokenizer
```

```
bhavana@Bhavana:~$ ./tokenizer
Enter a command: ADD 10 20

Tokens:
Token 1 -> Type: COMMAND, Value: ADD
Token 2 -> Type: NUMBER, Value: 10
Token 3 -> Type: NUMBER, Value: 20

Token stream is VALID.
```

Task 2

```
Token stream is VALID.
bhavana@Bhavana:~$ nano parser.c
```

```
GNU nano 2.7.1
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

struct ParseTree {
    char command[10];
    int left;
    int right;
};

int main() {

    char input[100];
    char command[10];
    int a, b;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    /* Handle empty command */
    if (strlen(input) <= 1) {
        printf("Error: Empty command.\n");
        return 0;
    }

    /* Parse input */
    int result = sscanf(input, "%s %d %d", command, &a, &b);

    if (result != 3) {
        printf("Syntax Error: Expected COMMAND NUMBER NUMBER.\n");
        return 0;
    }

    /* Syntax validation */
    if (strcmp(command, "ADD") != 0 &&
        strcmp(command, "SUB") != 0 &&
        strcmp(command, "MUL") != 0 &&
```

GNU nano 8.7.1

```
    strcmp(command, "MUL") != 0 &&
    strcmp(command, "DIV") != 0) {

    printf("Syntax Error: Invalid command.\n");
    return 0;
}

/* Division by zero */
if (strcmp(command, "DIV") == 0 && b == 0) {
    printf("Execution Error: Division by zero.\n");
    return 0;
}

/* Create parse tree */
struct ParseTree tree;

strcpy(tree.command, command);
tree.left = a;
tree.right = b;

printf("\nParse Tree:\n");
printf("      %s\n", tree.command);
printf("    /  \\\n");
printf("   %d   %d\n", tree.left, tree.right);

/* Execution structure */
printf("\nExecution Structure:\n");
printf("Command = %s\n", tree.command);
printf("Operand 1 = %d\n", tree.left);
printf("Operand 2 = %d\n", tree.right);

/* Execute */
int answer;

if (strcmp(command, "ADD") == 0)
    answer = a + b;
```

```
    if (strcmp(command, "ADD") == 0)
        answer = a + b;

    else if (strcmp(command, "SUB") == 0)
        answer = a - b;

    else if (strcmp(command, "MUL") == 0)
        answer = a * b;

    else
        answer = a / b;

    printf("\nExecution Result = %d\n", answer);

    return 0;
}
```

```
bhavana@Bhavana:~$ gcc parser.c -o parser
bhavana@Bhavana:~$ ./parser
```

```
Enter command: ADD 10 20

Parse Tree:
      ADD
     /  \
    10   20

Execution Structure:
Command = ADD
Operand 1 = 10
Operand 2 = 20

Execution Result = 30
bhavana@Bhavana:~$
```

Result

The command input was successfully tokenized and parsed. Whitespace and delimiters were handled correctly, tokens and arguments were identified, empty commands were detected, and the resulting execution structure was displayed successfully.